

ENCAPSULATION IN DART

Introduction

In this section, you will learn about **encapsulation in Dart** programming language with examples. Encapsulation is one of the important concepts of object-oriented programming. Before learning about dart encapsulation, you should have a basic understanding of the **class** and **object** in dart.

Encapsulation In Dart

In Dart, **Encapsulation** means **hiding data** within a library, preventing it from outside factors. It helps you control your program and prevent it from becoming too complicated.

What Is Library In Dart?

By default, every **.dart** file is a library. A library is a collection of functions and classes. A library can be imported into another library using the **import** keyword.

How To Achieve Encapsulation In Dart?

Encapsulation can be achieved by:

- Declaring the class properties as **private** by using **underscore(_)**.
- Providing public **getter** and **setter** methods to access and update the value of private property.

Info

Note: Dart doesn't support keywords like **public**, **private**, and **protected**. Dart uses **_** (underscore) to make a property or method private. The encapsulation happens at library level, not at class level.

Getter and Setter Methods

Getter and **setter** methods are used to access and update the value of private property.

Getter methods are used to access the value of private property. **Setter** methods are used to update the value of private property.

Example 1: Encapsulation In Dart

In this example, we will create a class named **Employee**. The class will have two private properties **_id** and **_name**. We will also create two public methods **getId()** and **getName()** to access the private properties. We will also create two public methods **setId()** and **setName()** to update the private properties.

```
class Employee {
  // Private properties
  int? _id;
  String? _name;

  // Getter method to access private property _id
  int getId() {
    return _id!;
  }
  // Getter method to access private property _name
  String getName() {
    return _name!;
  }
  // Setter method to update private property _id
  void setId(int id) {
    this._id = id;
  }
  // Setter method to update private property _name
  void setName(String name) {
    this._name = name;
  }
}

void main() {
  // Create an object of Employee class
  Employee emp = new Employee();
  // setting values to the object using setter
  emp.setId(1);
  emp.setName("John");

  // Retrieve the values of the object using getter
  print("Id: ${emp.getId()}");
  print("Name: ${emp.getName()}");
}
```

[➤ Show Output](#)

Run Online

Private Properties

Private property is a property that can only be accessed from same **library**. Dart does not have any keywords like **private** to define a private property. You can define it by prefixing an **underscore (_) to its name**.

Example 2: Private Properties In Dart

In this example, we will create a class named **Employee**. The class has one private property **_name**. We will also create a public method **getName()** to access the private property.

```
class Employee {  
  // Private property  
  var _name;  
  
  // Getter method to access private property _name  
  String getName() {  
    return _name;  
  }  
  
  // Setter method to update private property _name  
  void setName(String name) {  
    this._name = name;  
  }  
}  
  
void main() {  
  var employee = Employee();  
  employee.setName("Jack");  
  print(employee.getName());  
}
```

[➤ Show Output](#)

Run Online

Why Aren't Private Properties Private?

In the main method, if you write the following code, it will compile and run without any error. Let's see why it is happening.

```
class Employee {  
    // Private property  
    var _name;  
  
    // Getter method to access private property _name  
    String getName() {  
        return _name;  
    }  
  
    // Setter method to update private property _name  
    void setName(String name) {  
        this._name = name;  
    }  
}  
  
void main() {  
    var employee = Employee();  
    employee._name = "John"; // It is working, but why?  
    print(employee.getName());  
}
```

[Show Output](#)

Run Online

Reason

The reason is that using **underscore (_)** before a variable or method name makes it **library private** not **class private**. It means that the variable or method is only visible to the library in which it is declared. It is not visible to any other library. In simple words, library is one file. If you write the main method in a separate file, this will not work.

Solution

To see private properties in action, you must create a separate file for the class and import it into the main file.

Read-only Properties

You can control the properties's access and implement the encapsulation in the dart by using the read-only properties. You can do that by adding the **final** keyword before the properties declaration. Hence, you can only access its value, but you cannot change it.

Info

Note: Properties declared with the **final** keyword must be initialized at the time of declaration. You can also initialize them in the constructor.

```
class Student {
  final _schoolname = "ABC School";

  String getSchoolName() {
    return _schoolname;
  }
}

void main() {
  var student = Student();
  print(student.getSchoolName());
  // This is not possible
  //student._schoolname = "XYZ School";
}
```

[Show Output](#)

[Run Online](#)

Info

Note: You can also define **getter** and **setter** using **get** and **set** keywords. For more see this example below.

How To Create Getter and Setter Methods?

You can create getter and setter methods by using the **get** and **set** keywords. In this example below, we have created a class named **Vehicle**. The class has two private properties **_model** and **_year**. We have also created two getter and setter methods for each property. The getter and setter methods are named **model** and **year**. The getter and setter methods are used to access and update the value of the private properties.

```
class Vehicle {
  String _model;
  int _year;

  // Getter method
  String get model => _model;

  // Setter method
  set model(String model) => _model = model;

  // Getter method
  int get year => _year;

  // Setter method
  set year(int year) => _year = year;
}

void main() {
  var vehicle = Vehicle();
  vehicle.model = "Toyota";
  vehicle.year = 2019;
  print(vehicle.model);
  print(vehicle.year);
}
```

› Show Output

Run Online

Info

Note: In dart, any identifier like (class, class properties, top-level function, or variable) that starts with an underscore _ it is private to its library.

Why Encapsulation Is Important?

- **Data Hiding:** Encapsulation hides the data from the outside world. It prevents the data from being accessed by the code outside the class. This is known as data hiding.
- **Testability:** Encapsulation allows you to test the class in isolation. It will enable you to test the class without testing the code outside the class.
- **Flexibility:** Encapsulation allows you to change the implementation of the class without affecting the code outside the class.

- **Security:** Encapsulation allows you to restrict access to the class members. It will enable you to limit access to the class members from the code outside the library.

Video

Watch our video on encapsulation in Dart.